Taller: Integración y Tecnologías Emergentes — Android

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Este taller tiene dos secciones. La primera investiga qué es Android y cómo está hecho por dentro; la segunda crea la primera aplicación en Android Studio.

La investigación no se queda en la definición de diccionario. Donde una decisión de Android se parece a una que ya se tomó en MONK GRAIN —los permisos, el aislamiento entre usuarios, el ciclo de vida— se dice, porque es lo que convierte el dato en algo que se puede usar en el proyecto de AA2-EV02.

2. Objetivo

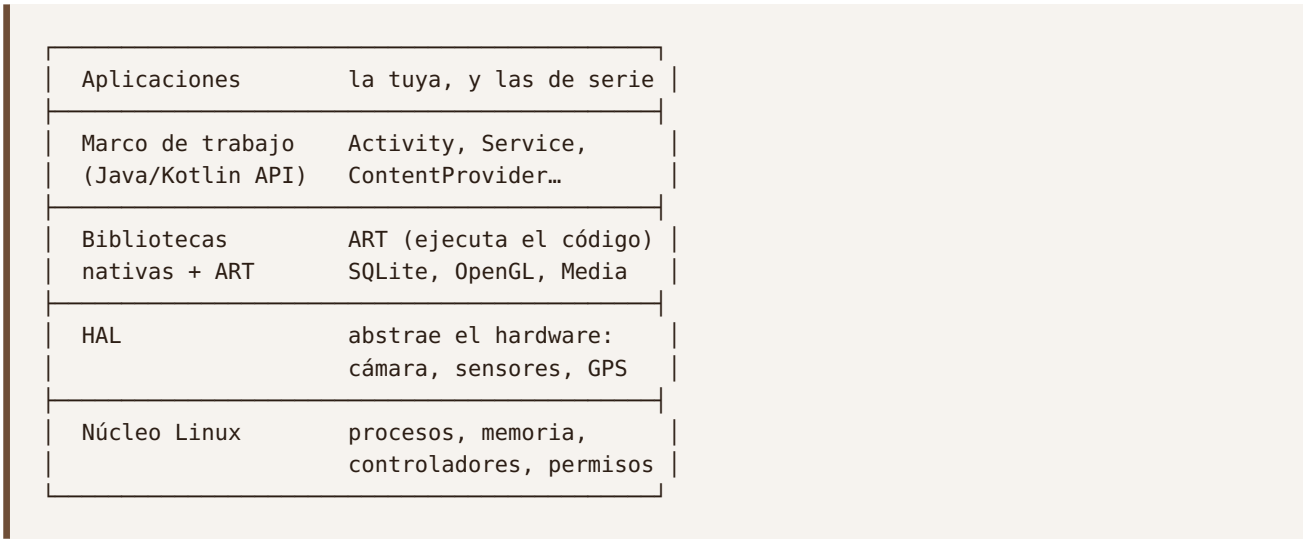
Investigar los conceptos fundamentales de Android y crear la primera aplicación con Android Studio, configurando el proyecto, una actividad, y ejecutándola hasta mostrar un mensaje en pantalla.

SECCIÓN 1 — Investigación

3. Qué es Android

Android es un **sistema operativo para dispositivos móviles** construido sobre el núcleo de Linux, desarrollado hoy por Google y la Open Handset Alliance. Nació en 2003 como una empresa independiente —Android Inc., de Andy Rubin— que Google compró en 2005; la primera versión pública es de 2008.

Lo que explica casi todo lo demás es que **Android no es Linux con otra cara**: es una pila de capas donde cada una resuelve un problema distinto.



La capa de abstracción del hardware (HAL) es la que permite que la misma aplicación funcione en un teléfono de gama baja y en uno de gama alta con cámaras distintas: la aplicación pide «una foto», no «esta cámara concreta».

4. Qué es un APK

APK es Android Package: el archivo que se instala en el dispositivo. Por dentro es un **ZIP** con una estructura fija.

Dentro del APK	Qué es
`AndroidManifest.xml`	La declaración: nombre, versión, permisos, actividades y cuál se abre primero
`classes.dex`	El código compilado a bytecode DEX, que es lo que ART ejecuta
`resources.arsc`	Los recursos compilados: textos, colores, medidas
`res/`	Imágenes y diseños de pantalla
`lib/`	Bibliotecas nativas, una carpeta por arquitectura
`META-INF/`	**La firma digital**

Todo APK va firmado, y la firma hace dos cosas. Identifica a quien lo publicó —no por su nombre, sino porque la clave es siempre la misma— y permite que el sistema compruebe que el archivo no fue modificado después. De ahí sale una regla con consecuencias prácticas: **una actualización solo se instala sobre una aplicación firmada con la misma clave**. Perder la clave de firma significa no poder volver a actualizar la aplicación nunca; hay que publicarla con otro identificador y pedirle a cada usuario que la instale de cero.

Desde 2021, Google Play recibe en cambio un **AAB** (Android App Bundle), y es Play quien genera un APK a medida de cada dispositivo. La diferencia importa: el AAB se sube a la tienda, el APK se instala. Para una aplicación interna como la de AA2-EV02, que se instala a mano, el APK sigue siendo el formato.

5. El Android SDK

El **SDK** (Software Development Kit) es el conjunto de piezas que hacen falta para construir una aplicación.

Pieza	Para qué
SDK Platforms	Las API de cada versión de Android. Se compila contra una
Build Tools	`aapt2` compila recursos, `d8` convierte a DEX, `apksigner` firma
Platform Tools	`adb`, el puente para instalar y depurar en el dispositivo
Emulator	Dispositivos virtuales para probar sin teléfono
Gradle	El sistema de construcción: dependencias, variantes, firma

5.1 Tres números que se confunden todo el tiempo

Nivel de API	Qué significa	Qué decide
`minSdk`	La versión más antigua que puede instalarla	A cuánta gente llegas
`targetSdk`	La versión contra la que se probó	Qué comportamientos nuevos se te aplican
`compileSdk`	Las API disponibles al compilar	Qué puedes escribir

Bajar `minSdk` amplía el público y obliga a comprobar en tiempo de ejecución si una función existe. Subir `targetSdk` es obligatorio para publicar en Play y activa restricciones nuevas: una aplicación con `targetSdk` viejo conserva permisos que ya no se conceden. Es la tensión central de mantener una aplicación Android.

6. Lenguajes

Lenguaje	Situación
Kotlin	El recomendado por Google desde 2019. Tipos que distinguen lo que puede ser nulo de lo que no, corrutinas para el trabajo en segundo plano, menos código repetido
Java	Sigue funcionando. Millones de líneas en producción, y es el que se ha usado en GA7
C / C++ (NDK)	Solo para lo que necesita velocidad: motores de juego, procesamiento de señal
Dart, JavaScript	Multiplataforma (Flutter, React Native). No son Android nativo

Los dos primeros conviven en el mismo proyecto porque **ambos compilan al mismo bytecode DEX**. Una clase Kotlin puede llamar a una Java y al revés, sin puente ni conversión.

> **Qué se usará en AA2-EV02, y por qué.** El proyecto móvil de MONK GRAIN se escribirá en **Kotlin**. El motivo no es que sea lo recomendado: es que el tipo que distingue lo nulo de lo no nulo resuelve en el compilador justo el error que más aparece al consumir una API —un campo que llega vacío y nadie lo previó—. En Java ese error aparece en el teléfono del usuario; en Kotlin, al compilar.

7. Entornos de desarrollo

IDE	Estado
Android Studio	El oficial, sobre IntelliJ IDEA. Trae SDK, emulador, editor de diseño, analizadores y perfilador
IntelliJ IDEA	Con el complemento de Android. Mismo motor, menos herramientas específicas
Visual Studio Code	Posible con extensiones, pero sin editor de diseño ni emulador integrado
Eclipse + ADT	**Descontinuado** en 2015. Aparece en tutoriales viejos y ya no se usa

8. Android multiusuario

Android soporta **varios usuarios en un mismo dispositivo**, cada uno con sus aplicaciones, sus datos y sus cuentas. Hay tres figuras distintas:

Figura	Qué es
Usuario completo	Un espacio independiente. Instala sus aplicaciones, no ve las de los demás
Perfil restringido	Comparte las aplicaciones del dueño con límites. Pensado para menores
Perfil de trabajo	Separa lo laboral de lo personal en el mismo teléfono. La empresa administra solo ese perfil

El mecanismo es sencillo y contundente: **cada usuario tiene su propio directorio de datos** (`/data/user/0/`, `/data/user/10/`…) y el núcleo impide que un proceso lea el del otro. No es una cortina en la interfaz, es un permiso del sistema de archivos.

Para quien desarrolla, la consecuencia práctica es que **la misma aplicación puede estar instalada dos veces con datos distintos**. Una aplicación que guarde una sesión suponiendo que hay un solo usuario se comportará de forma extraña cuando la abra el segundo. Es el mismo error de fondo que guardar estado global en un servidor web que atiende a varias personas.

9. Mínimo privilegio

Es el principio que atraviesa toda la seguridad de Android: **cada aplicación recibe lo mínimo para funcionar, y nada más**.

9.1 Cómo se implementa

Mecanismo	Cómo funciona
Un UID por aplicación	Android aprovecha los usuarios de Linux: cada aplicación instalada recibe su propio identificador de usuario, y sus archivos pertenecen a ese UID. Una aplicación no puede leer los datos de otra porque el sistema de archivos no se lo permite
Proceso aislado	Cada aplicación corre en su propio proceso con su propia instancia de ART. Si una se cae, no arrastra a las demás
Permisos declarados	Lo que necesita acceso fuera de su caja se declara en el manifiesto. Lo no declarado, no existe para esa aplicación
Permisos peligrosos en tiempo de ejecución	Desde Android 6, cámara, ubicación, contactos y micrófono se piden al usar, no al instalar, y el usuario puede negarlos o concederlos solo esta vez
SELinux	Control de acceso obligatorio que limita incluso a los procesos del sistema. Una aplicación comprometida no hereda todo el dispositivo
Alcance del almacenamiento	Desde Android 10, una aplicación ve sus archivos y lo que el usuario elija, no la tarjeta entera

> **Es el mismo principio que gobierna el backend de MONK GRAIN.** Allí los endpoints no preguntan «¿eres administrador?» sino «¿tienes el permiso `product.create`?», y hay 24 permisos en vez de cuatro roles con poderes amplios. La idea de fondo es idéntica: conceder lo mínimo y nombrarlo. La diferencia es quién concede —el usuario del teléfono en Android, un administrador en la plataforma—.

9.2 Lo que esto significa al escribir la aplicación

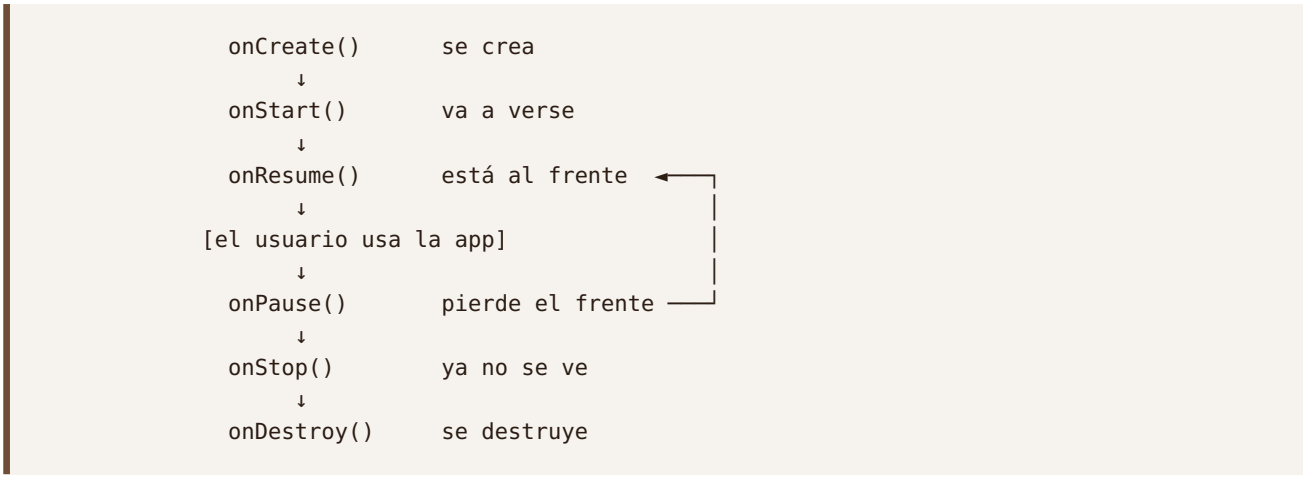
- **Pedir un permiso cuesta usuarios.** Cada diálogo es una oportunidad de que digan que no y de que desconfíen. La aplicación de AA2-EV02 solo necesitará `INTERNET`, que ni siquiera abre diálogo.
- **Hay que programar para el «no».** Un permiso puede negarse, o concederse «solo esta vez». La aplicación tiene que seguir siendo útil sin él, no quedarse en una pantalla en blanco.
- **Los datos sensibles no van en almacenamiento compartido.** Un token de sesión va en el almacenamiento privado de la aplicación o, mejor, cifrado.

10. Componentes esenciales de una aplicación

Son cuatro, y todos tienen algo en común: **el sistema los crea y los destruye, no la aplicación**. Esa inversión explica casi todos los errores de principiante en Android.

Componente	Qué es	Ejemplo en MONK GRAIN móvil
Activity	Una pantalla con la que el usuario interactúa	La lista de lotes; el detalle de un lote
Service	Trabajo sin interfaz, que sigue aunque se cierre la pantalla	Subir la foto de un lote mientras el caficultor hace otra cosa
Broadcast Receiver	Reacciona a avisos del sistema o de otras aplicaciones	Volver a intentar el envío cuando regrese la conexión
Content Provider	Comparte datos con otras aplicaciones, de forma controlada	No se usará: los datos del lote no se comparten fuera

10.1 El ciclo de vida de una Activity, y por qué importa



Lo que convierte esto en algo más que un diagrama es que **el sistema puede destruir una Activity para recuperar memoria y recrearla después**, y también lo hace al girar el teléfono. Si el estado vive en una variable de la Activity, girar la pantalla lo borra.

Es el error clásico de Android: la aplicación funciona, el usuario gira el teléfono, y el formulario a medio llenar desaparece. Se evita guardando el estado donde sobreviva —hoy, un `ViewModel`—, no recordando poner el teléfono derecho.

10.2 El manifiesto y los intents

`AndroidManifest.xml` declara los componentes. Lo que no está declarado, **no existe para el sistema**: una Activity sin su entrada no se puede abrir, aunque la clase esté compilada dentro del APK.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Lo único que necesita la app de MONK GRAIN: hablar con la API.
         INTERNET es un permiso normal, no peligroso: se concede al
         instalar y no abre ningún diálogo al usuario. -->
    <uses-permission android:name="android.permission.INTERNET" />

    <application android:label="MONK GRAIN">
        <activity
            android:name=".MainActivity"
            android:exported="true">

            <!-- Este filtro es lo que pone el icono en el lanzador.
                 Sin él la app se instala y no hay forma de abrirla. -->
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Los componentes se comunican con **intents**, que son mensajes. Un intent explícito nombra a quién va dirigido; uno implícito describe lo que se quiere —«abrir esta dirección»— y el sistema ofrece quién puede hacerlo. El implícito es lo que permite que compartir una foto muestre la lista de aplicaciones capaces de recibirla, sin que ninguna conozca a las otras.

SECCIÓN 2 — La primera aplicación

Esta sección se hace en el computador del aprendiz: instalar Android Studio, crear el proyecto, ejecutarlo y capturar el proceso. Lo que sigue es el procedimiento paso a paso con lo que debe verse en cada captura.

11. Instalar Java y Android Studio

Android Studio trae su propio JDK, así que no hace falta instalar Java aparte. En este equipo ya está, de los módulos de GA7:

```
$ java -version
openjdk version "17.0.20.1" 2026-08-18
```

```
# Android Studio en Linux Mint / Ubuntu
$ sudo snap install android-studio --classic

# o descargando el archivo desde developer.android.com/studio
$ tar -xzf android-studio-*.tar.gz -C ~/
$ ~/android-studio/bin/studio.sh
```

La primera vez abre un asistente que descarga el SDK, las herramientas de construcción y una imagen de emulador. Son varios gigabytes: conviene no hacerlo con prisa.

Captura 1 — Android Studio instalado

La pantalla de bienvenida, con el número de versión visible

Captura 2 — El SDK Manager

Tools → SDK Manager, mostrando la plataforma descargada (API 34 o superior) y las Build Tools

12. Crear el proyecto

Campo	Valor	Por qué
Plantilla	**Empty Views Activity**	Crea una Activity con su diseño en XML, que es lo que pide el taller
Name	`Hola MONK GRAIN`	—
Package name	`co.monkgrain.hola`	Mismo prefijo que los módulos Java de GA7
Language	**Kotlin**	§6
Minimum SDK	**API 24** (Android 7.0)	Cubre más del 95 % de los dispositivos en uso
Build configuration	Kotlin DSL (`build.gradle.kts`)	Es lo que Android Studio propone hoy

Captura 3 — El asistente de nuevo proyecto

El diálogo con el nombre, el paquete, Kotlin y API 24 escritos

12.1 Lo que crea

```
HolaMonkGrain/
├── app/
│   ├── build.gradle.kts          dependencias y versiones de SDK
│   └── src/main/
│       ├── AndroidManifest.xml   la declaración
│       ├── java/co/monkgrain/hola/
│       │   └── MainActivity.kt   la actividad principal
│       └── res/
│           ├── layout/activity_main.xml   el diseño
│           └── values/strings.xml         los textos
├── build.gradle.kts
└── settings.gradle.kts
```

> **Los textos van en `strings.xml`, no dentro del diseño.** No es burocracia: es lo que permite traducir la aplicación añadiendo `values-en/strings.xml` sin tocar una sola pantalla. Es la misma idea que `messages.properties` en el módulo de Spring de GA7-AA3.

13. La actividad principal

`MainActivity.kt`:

```
package co.monkgrain.hola

import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

/**
 * La primera pantalla de la aplicación.
 *
 * Hereda de AppCompatActivity y no de Activity a secas: AppCompat trae las
 * funciones nuevas a las versiones viejas de Android, que es lo que permite
 * poner minSdk en 24 sin renunciar a lo de hoy.
 */
class MainActivity : AppCompatActivity() {

    /**
     * El sistema llama a este método al crear la pantalla, y también al
     * recrearla —por ejemplo, al girar el teléfono—. Por eso todo lo que
     * dibuja va aquí y no en el constructor: el constructor corre una vez,
     * `onCreate` corre cada vez que hay que volver a dibujar.
     *
     * `savedInstanceState` trae el estado guardado si es una recreación, y
     * es `null` la primera vez. De ahí el tipo `Bundle?`, con interrogante:
     * Kotlin obliga a decidir qué pasa cuando no hay nada.
     */
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Conecta la pantalla con su diseño en XML.
        setContentView(R.layout.activity_main)

        // R es una clase que Android genera al compilar los recursos: por eso
        // un error en el XML aparece aquí como «no se encuentra R».
        val mensaje = findViewById<TextView>(R.id.mensaje)
        mensaje.text = getString(R.string.saludo)
    }
}
```

`res/layout/activity_main.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- El identificador es lo que permite encontrarlo desde el código
         con findViewById. Sin id, la vista existe y es inalcanzable. -->
    <TextView
        android:id="@+id/mensaje"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="22sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

`res/values/strings.xml`:

```
<resources>
    <string name="app_name">Hola MONK GRAIN</string>
    <string name="saludo">¡Hola! Soy Eliana Chacon</string>
</resources>
```

> **Por qué `sp` y no `dp` para el texto.** `dp` es independiente de la densidad de la pantalla; `sp` además respeta el tamaño de letra que el usuario configuró en el sistema. Usar `dp` en texto ignora a quien necesita la letra grande para leer.

14. Compilar y ejecutar

```
# desde Android Studio: el botón ▶ Run

# o desde la terminal, en la carpeta del proyecto:
$ ./gradlew assembleDebug
BUILD SUCCESSFUL

# el APK queda en:
app/build/outputs/apk/debug/app-debug.apk

# instalarlo en un dispositivo o emulador conectado:
$ adb install app/build/outputs/apk/debug/app-debug.apk
```

Captura 4 — El proyecto abierto

Android Studio con `MainActivity.kt` en el editor y el árbol de archivos a la izquierda

Captura 5 — El dispositivo virtual

Tools → Device Manager con el emulador creado

Captura 6 — La compilación

La ventana **Build** con `BUILD SUCCESSFUL`

Captura 7 — La aplicación en marcha

El emulador o el teléfono mostrando el mensaje **«¡Hola! Soy Eliana Chacon»** centrado en la pantalla

Captura 8 — El APK generado

El archivo `app-debug.apk` en `app/build/outputs/apk/debug/`, con su tamaño visible

15. Problemas frecuentes la primera vez

Síntoma	Causa	Qué hacer
Gradle se queda descargando para siempre	La primera construcción baja el wrapper y las dependencias	Esperar. Solo ocurre la primera vez
`SDK location not found`	Falta `local.properties`	Crearlo con `sdk.dir=/home/usuario/Android/Sdk`. **No se versiona**: cada máquina tiene su ruta
El emulador no arranca	Virtualización desactivada en la BIOS	Activar VT-x o AMD-V, o usar un teléfono real por USB
`adb`: device unauthorized	No se aceptó la depuración en el teléfono	Aceptar el diálogo que aparece en la pantalla del dispositivo
«No se encuentra R»	Hay un error en algún XML	El error real está en el XML; `R` no se genera hasta que compile

> **Lo que te falta hacer a ti**
>
> **Toda la Sección 2 en tu computador**: instalar Android Studio, crear el proyecto con los valores de §12, escribir los tres archivos de §13, ejecutarlo y capturar las **8 capturas** señaladas.
>
> **El APK** resultante (`app-debug.apk`) va dentro del RAR de la entrega, junto a este PDF y su HTML.
>
> El código de §13 está listo para copiar y pegar; cambia el nombre del saludo si quieres. La Sección 1 no requiere nada tuyo.

16. Conclusión

Android es Linux aprovechado de una forma concreta: cada aplicación es un usuario del sistema, con sus archivos protegidos por los permisos del sistema de archivos y no por una cortina en la interfaz. De ahí salen el aislamiento entre aplicaciones, el soporte multiusuario y el mínimo privilegio — tres cosas que parecen separadas y son el mismo mecanismo visto desde ángulos distintos.

Para quien desarrolla, lo que más cambia respecto de una aplicación web es que **el sistema manda**: crea y destruye las pantallas cuando le conviene, y concede o niega los permisos según lo que diga el usuario. Programar para Android es, en buena medida, programar para que eso no rompa nada.